

Tri bitonique & Minage de blockchain

Un même algorithme, trois paradigmes de langage
(OCaml, Lean 4, OpenQASM)

KAMDOM NGUETCHESSI MERVEILLE PRISCA

Matricule : 23V2060

Université de Yaoundé I – Département d'Informatique

INF3421 : Langages Formels & Compilation

Supervision : Dr. MESSI NGUELE Thomas

Année académique 2025–2026

De quoi parle ce travail ?

Deux algorithmes, choisis pour ce qu'ils ont en commun

Le fil conducteur

- Un algorithme dont la **structure de calcul ne dépend pas des données**.
- **Tri bitonique** : la suite des comparaisons est fixée d'avance $\Rightarrow$ un vrai circuit.
- **Minage (Proof of Work)** : recherche par force brute $\Rightarrow$ ce que Grover accélère.

L'idée pédagogique

- Écrire le *même* algorithme dans trois langages très différents.
- Comprendre qu'une bonne compréhension de la structure d'un algorithme le rend traduisible partout.
- Objectif : la clarté avant la performance.

Petit rappel : comment « fonctionne » un langage ?

Les grandes familles, pour situer nos trois langages

Selon la façon de décrire le calcul

- **Impératif** : on donne une suite d'instructions qui modifient l'état (« fais ceci, puis cela »).
- **Déclaratif** : on décrit *ce que l'on veut*, pas comment l'obtenir (fonctionnel, logique, formel).

Selon la façon d'exécuter

- **Compilé** : le code source est traduit une fois en langage machine, puis exécuté (rapide).
- **Interprété** : un programme lit et exécute le code ligne à ligne (souple, plus lent).

La chaîne classique

Code source → analyse lexicale → analyse syntaxique → vérification des types → génération de code → exécution.
C'est exactement le sujet du cours INF3421 : transformer un texte en un calcul.

Nos trois langages, et pourquoi eux

OCaml – l'exécutable

Langage **fonctionnel** qui autorise aussi le style **impératif** (tableaux mutables). **Compilé**, rapide, avec un système de types puissant. Ici : implémentation qui tourne réellement et que l'on peut tester.

Lean 4 – le rigoureux

Langage à **typage dépendant** : le type peut exprimer une propriété mathématique. Style **fonctionnel pur** et **déclaratif**. Ici : on énonce formellement ce que « bien trier » ou « chaîne valide » veut dire, et on peut le *prouver*.

OpenQASM – le circuit

Langage de **description de circuits quantiques** : on assemble des portes logiques réversibles. Ici : on traduit l'algorithme en portes, pour voir jusqu'où il « descend » vers le matériel.

Algorithme 1 : le tri bitonique

Un réseau de tri câblé une fois pour toutes

- Une suite **bitonique** croît puis décroît (ex. 1, 3, 5, 4, 2).
- Idée de Batcher (1968) : couper en deux, comparer/échanger a_i avec $a_{i+n/2} \Rightarrow$ deux sous-suites bitoniques, l'une $\leq$ l'autre.
- On applique récursivement (*diviser pour régner*).
- Coût : $O(n \log^2 n)$ comparaisons **fixées d'avance** $\Rightarrow$ parallélisable, traduisible en circuit.

```
(* La brique de base, en OCaml *)
let compare_and_swap a i j dir =
  let doit_echanger = match dir with
    | Asc   -> a.(i) > a.(j)
    | Desc  -> a.(i) < a.(j)
  in
  if doit_echanger then begin
    let tmp = a.(i) in
    a.(i) <- a.(j); a.(j) <- tmp
  end
```

Le tri bitonique dans les trois langages

OCaml (impératif)

- Tableau muté *en place*.
- Testé sur 8 et 16 éléments, ordre croissant *et* décroissant.
- Contrainte : taille = puissance de 2.

Lean 4 (fonctionnel pur)

- Mêmes fonctions, mais sur des *listes immuables*.
- On énonce : le résultat est *trié* et c'est une *permutation* de l'entrée.
- Preuves laissées en sorry (spécification > preuve).

OpenQASM (circuit)

- Échange conditionnel = porte de Fredkin (SWAP contrôlé).
- Condition calculée sur un qubit ancilla (Toffoli), puis « défaite ».
- Exemple à 4 qubits, mêmes étapes que le code OCaml.

À retenir

La *structure* (comparer, échanger, fusionner) est la même partout : seule la manière de l'exprimer change.

Algorithme 2 : le minage de blockchain (Proof of Work)

Chercher une aiguille dans une botte de foin numérique

- Un bloc contient : données, hash du bloc précédent, et un **nonce**.
- **Effet avalanche** : changer un bit d'entrée change $\approx$ la moitié des bits du hash.
- **Miner** = trouver un nonce tel que le hash commence par d zéros.
- Aucun raccourci : il faut essayer $\approx 16^d$ nonces $\Rightarrow$ coût dissuasif, sécurité.

Résultat obtenu (OCaml, MD5, $d = 4$)

Bloc 0 miné en 83692 essais

Bloc 1 miné en 68847 essais

Chaîne valide ? true

Après fraude sur le bloc 1 ? **false**

Toute modification a posteriori est *détectée* : le hash stocké ne colle plus au contenu.

Le minage dans les trois langages

OCaml (impératif)

- Hachage MD5 (module standard, sans dépendance).
- `miner` incrémente le nonce en boucle.
- `chaîne_valide` vérifie intégrité + chaînage.

Lean 4 (formel)

- Hachage modélisé par une fonction *abstraite*.
- Théorème `fraude_detectee` : *si* le hachage est injectif, toute fraude est détectée.
- Sépare hypothèse de sécurité et fait démontrable.

OpenQASM (Grover)

- Recherche non structurée $\Rightarrow$ Grover : $O(\sqrt{N})$ au lieu de $O(N)$.
- Exemple jouet : 2 qubits, un nonce marqué, un diffuseur.
- Limite assumée : pas de vrai oracle SHA-256.

Honnêteté scientifique

Les limites (petites tailles, uncompute simplifié, pas de SHA-256 quantique, ère NISQ) sont *explicitées*, pas masquées.

Conclusion comparative

Trois langages, trois façons de « dire » le même calcul

	OCaml	Lean 4	OpenQASM
Paradigme	Impératif / fonctionnel	Fonctionnel pur, formel	Description de circuits
Exécution	Compilé, rapide	Compilé + vérificateur	Simulé / matériel quantique
Ce qu'on gagne	Ça tourne, c'est testé	On <i>spécifie</i> et on <i>prouve</i>	On voit la structure physique
Ce qu'on perd	Correction non prouvée	Preuve complète coûteuse	Réaliste seulement en jouet

- **Le message central** : un même algorithme s'exprime en impératif, fonctionnel pur, formel ou quantique.
- C'est la compréhension *profonde* de sa structure – indépendante de la syntaxe – qui rend ces traductions possibles et correctes.

Merci de votre attention.

Questions & discussion

KAMDOM NGUETCHESSI MERVEILLE PRISCA – 23V2060
INF3421 : Langages Formels & Compilation – UY1 – 2025–2026